# Technical Solution Submission

**SapientAI — lightweight by design**

## Quick Access

For fast review, the three main public entry points are:

- GitHub repository: https://github.com/KpihX/insight
- Documentation / presentation: https://kpihx.github.io/insight-presentation/#/README.md
- Live web application: https://insight-6roy3g9xb-kamdem-ivanns-projects.vercel.app?_vercel_share=Uyqvz5abqLHvz3rUVbiirCec9NXIzmKN

## 1. Problem First

School communication is fragmented across multiple channels such as email, messaging, and institutional portals.

This creates a practical problem for teachers:

- important information arrives in different places,
- action-required items are easy to miss,
- schedule-impacting messages are mixed with routine communication,
- and timetable changes are often interpreted manually, late, or inconsistently.

Our project addresses this problem with a simple principle:

```
incoming school communication
-> structured understanding
-> teacher-facing action surface
-> human validation for schedule changes
```

The goal is not to automate blindly.

The goal is to help a teacher detect what matters, understand it quickly, and stay in control when a communication should affect the timetable.

## 2. Product Positioning

Our product positioning can be summarized in one sentence:

```
lightweight by design
```

This slogan is not cosmetic. It reflects a real product and architecture choice.

Many school-facing administrative tools become heavy very quickly:

- too many screens,
- too many manual clicks,
- too much friction before reaching the useful action,
- and too much complexity for routine teaching workflows.

Our position is different.

We are not trying to replace the entire school information system with a large, heavy platform.

We are building a light operational layer that sits on top of fragmented communication and turns it into:

```
clarity
-> prioritization
-> suggested action
-> validated schedule update when needed
```

That is what `lightweight by design` means in practice:

- small visible surface,
- direct paths to the important information,
- fast comprehension,
- low friction for the teacher,
- and strong human control when the impact is high.

## 3. What The Solution Does

The solution combines:

- a backend orchestration layer built with n8n,
- a frontend teacher-facing application deployed on Vercel,
- an AI classification step that extracts operational meaning from messages,
- and a human-in-the-loop validation flow for any schedule-impacting event.

In practice, when a message arrives:

```
message arrives
-> backend normalizes the source
-> AI classifies the message
-> backend decides whether it is informational or schedule-impacting
-> frontend surfaces the result to the teacher
```

Two main behaviors are currently demonstrated:

### A. Non-time informational event

If the message does not require a timetable change:

```
message received
-> AI generates a short summary
-> frontend shows a notification
```

### B. Time or meeting-related event

If the message implies a meeting or a timetable-impacting event:

```
message received
-> AI extracts a proposed calendar patch
-> frontend opens a dialog
-> teacher reviews date / time / room / title
```

```
-> teacher validates
-> event appears on the timetable
```

This is a strict human-in-the-loop approach:

```
AI proposes
teacher validates
system updates the visible schedule
```

## 4. Why This Is Different

The originality of the project is not only technical. It is also in the operational philosophy.

### A. Human-in-the-loop by default

We do not let the system silently rewrite a teacher's schedule.

Instead:

```
AI reads
-> AI structures
-> AI proposes
-> human decides
```

This is especially important for meetings, schedule changes, and any event that affects the teacher agenda.

### B. Proactive suggestion instead of passive storage

Many tools store information. Fewer tools actively help the user act on it.

Our system is intentionally proactive:

```
message arrives
-> summary is surfaced
-> important events are highlighted
-> schedule-impacting items trigger a proposal
```

The user is not asked to manually inspect everything first.

The system brings forward what deserves attention.

### C. Lightweight instead of heavy workflow friction

A heavy alternative often works like this:

```
open communication tool
-> read full message
-> copy useful information mentally
-> open another platform
-> create a calendar item manually
-> return to inbox
-> remember follow-up later
```

Our approach is:

```
message arrives
-> AI extracts the operational meaning
-> the interface surfaces the right action shape
-> the teacher validates once
```

This is the core differentiation.

We reduce cognitive and operational friction without removing human control.

## 5. Simplified Architecture

The architecture is intentionally simple to understand:

```
Frontend
SapientAI web application
-> deployed on Vercel

Backend
n8n workflows
-> ingestion
-> normalization
-> AI classification
-> persistence
-> API for the frontend
```

More concretely:

```
Email / messaging / portal input
-> n8n backend
-> normalized event
-> AI classification
-> API response
-> SapientAI frontend
```

Current public stack:

- Frontend: Vercel
- Backend orchestration: n8n
- Public documentation: GitHub Pages
- Source code: GitHub

## 6. Wellbeing Index

The application also includes a `Wellbeing Index`.

This is not a black-box score. It is a transparent workload indicator built from the live teacher-facing event stream.

It uses signals such as:

- pending action-required items,
- urgency,
- importance,
- and deadline-related pressure.

Its purpose is simple:

```
communication pressure
-> visible workload signal
-> teacher can understand current load at a glance
```

This contributes to the product philosophy:

```
not only collect messages
-> help the teacher understand operational pressure
```

So the interface does not only react to one message at a time.

It also gives a lightweight synthetic reading of the teacher's current state.

## 7. Public Links

### Source Code

- GitHub repository: https://github.com/KpihX/insight

### Technical Documentation

- Documentation / presentation: https://kpihx.github.io/insight-presentation/#/README.md

### Live Web Application

- SapientAI web application: https://insight-6roy3g9xb-kamdem-ivanns-projects.vercel.app?_vercel_share=Uyqvz5abqLHvz3rUVbiirCec9NXIzmKN

For additional technical details, the documentation mirror above is the main public entry point.

## 8. How To Take The Project In Hand

### Step 1. Open the live application

Open the web application:

- https://insight-6roy3g9xb-kamdem-ivanns-projects.vercel.app?_vercel_share=Uyqvz5abqLHvz3rUVbiirCec9NXIzmKN

For demo simplicity, the site is already opened on a teacher test profile:

```
Teacher test account: Sarah Lee
```

This means the evaluator does not need to configure anything before testing the experience.

### Step 2. Send a test email

To test the email flow, simply send an email to:

```
nextgenproject373@gmail.com
```

This mailbox is associated with the teacher test profile used in the application.

### Step 3. Wait for the IMAP trigger

Important note:

```
the IMAP trigger is not instantaneous
```

Please allow some delay before the message appears in the interface.

In practice, this may take:

- tens of seconds,
- and sometimes a few minutes depending on trigger latency.

So the correct expectation is:

```
send email
-> wait a little
-> check the live application
```

## 9. Two Email Scenarios To Test

Below are two ready-to-use email templates.

They demonstrate the two main product behaviors:

- non-time event,
- time / scheduling event.

### Scenario A. Non-time event

Expected result:

```
AI creates a short operational summary
-> frontend shows a notification
-> no timetable dialog is opened
```

Suggested email:

#### Subject

```
Updated attendance policy reminder
```

#### Body

```
Hello Sarah,

Please note that the updated attendance reporting policy is now in effect.

Teachers must submit attendance incidents before 5:00 PM on the same day whenever possible.

No action is required immediately, but please keep this change in mind for future reports.

Best regards,
David Brown
Administrative Office
```

Expected interpretation:

```
administrative information
-> summarized by AI
-> surfaced as a non-time notification
```

#### Scenario B. Time or meeting-related event

Expected result:

```
AI detects a schedule-impacting event
-> frontend opens a validation dialog
-> teacher reviews the proposal
-> teacher validates
-> event appears in the agenda
```

Suggested email:

#### Subject

```
Parent meeting scheduled for Tim Doe on Tuesday, March 17 at 4:00 PM
```

#### Body

```
Hello Sarah,

Jane Doe has requested a parent meeting regarding Tim Doe.

It has been scheduled for Tuesday, March 17, 2026 from 4:00 PM to 5:00 PM in Guidance Room B1

Please keep that slot available and bring Tim Doe's attendance notes.

Best regards,
David Brown
Administrative Office
```

Expected interpretation:

```
meeting detected
-> AI proposes a calendar patch
-> human checks title / time / room
-> validation inserts the event into the visible schedule
```

### 10. Human-In-The-Loop Design

One of the key design decisions of the project is that timetable changes are never applied blindly.

The system always follows this principle:

```
communication suggests a schedule change
-> AI extracts a structured proposal
-> teacher reviews it
```

```
-> teacher confirms it
-> timetable is updated
```

This is important because:

- schedule changes are high-impact,
- extracted dates and times must remain reviewable,
- and the final decision must remain under human control.

This design principle is central to our positioning:

```
assistive AI
not autonomous hidden automation
```

## 11. Messaging Case

We also support a messaging-based scenario built on the same principle.

In that case:

```
message arrives
-> backend normalizes it
-> AI classifies it
-> frontend surfaces either a summary or a schedule proposal
```

For security and operational reasons, this messaging path is currently connected to a contact controlled by a member of the team.

Therefore:

- this messaging case will be demonstrated live during the presentation,
- a QR code will be shown during the live demo,
- the audience will be able to send a message to the demonstration number,
- and the message will be processed through the same logic as the email scenario.

So the evaluation approach is:

```
email testing
-> self-service from the public instructions

messaging testing
-> shown live during the presentation
```

## 12. Why n8n + Vercel

The technical stack also reflects our product philosophy.

### Backend: n8n

We use n8n as an orchestration backbone because it is well suited for:

- multi-source ingestion,
- normalization pipelines,
- AI integration,
- storage and API routing,

- and fast iteration on event-driven logic.

#### Frontend: Vercel

We use Vercel for the visible product layer because it gives us:

- fast deployment,
- low-friction public access,
- and a clean static hosting surface for the teacher-facing application.

Together, this creates a clear split:

```
n8n
-> operational intelligence and orchestration

Vercel
-> lightweight visible experience
```

This combination is consistent with the overall design:

```
powerful backend orchestration
-> lightweight teacher-facing surface
```

### 13. Recommended Evaluation Path

For the fastest and clearest understanding of the project, we recommend this order:

#### 1. Read the public documentation

- https://kpihx.github.io/insight-presentation/#/README.md

#### 2. Open the live application

- https://insight-6roy3g9xb-kamdem-ivanns-projects.vercel.app?_vercel_share=Uyqvz5abqLHvz3rUVbiirCec9NXIzmKN

#### 3. Send the non-time email

Observe:

```
notification + AI summary
```

#### 4. Send the time-related email

Observe:

```
dialog + proposed agenda insertion + human validation
```

#### 5. See the messaging case during the live presentation

Observe:

```
same product logic through another communication channel
```

## 14. Closing Summary

This technical solution is built around one practical educational workflow problem:

```
too many fragmented messages
-> not enough structured visibility
-> too much manual interpretation for schedule-impacting communication
```

Our answer is:

```
n8n backend for orchestration
-> AI for structured understanding
-> SapientAI frontend for teacher-facing action
-> human validation for timetable changes
```

This keeps the system:

- useful,
- understandable,
- demonstrable,
- and operationally safe.

Most importantly, it keeps the product aligned with its core identity:

```
SapientAI
lightweight by design
```

That means:

- less friction,
- more clarity,
- proactive support,
- measurable workload visibility,
- and human control where it matters most.